Zero-Downtime Database Migrations

expand-contract, lock timeouts and safe backfills

Watch the 3-min video

In one line: version every schema change with a migration tool, avoid long locks (lock timeouts, `CREATE INDEX CONCURRENTLY`), and split breaking changes into expand, dual write, backfill, switch reads, contract, so old and new code always work together.

1 Why a simple rename causes an outage

You want to rename a column from `name` to `full_name`. One `ALTER TABLE … RENAME COLUMN`, then deploy. But a **rolling deploy** replaces servers a few at a time. For several minutes, half the fleet still runs the old code, which reads `name`. Those servers throw errors, and checkout breaks for ten minutes.

The root cause: the schema changed in one instant, but the code changes gradually. Any schema change must work with **both** the old and the new version of the app.

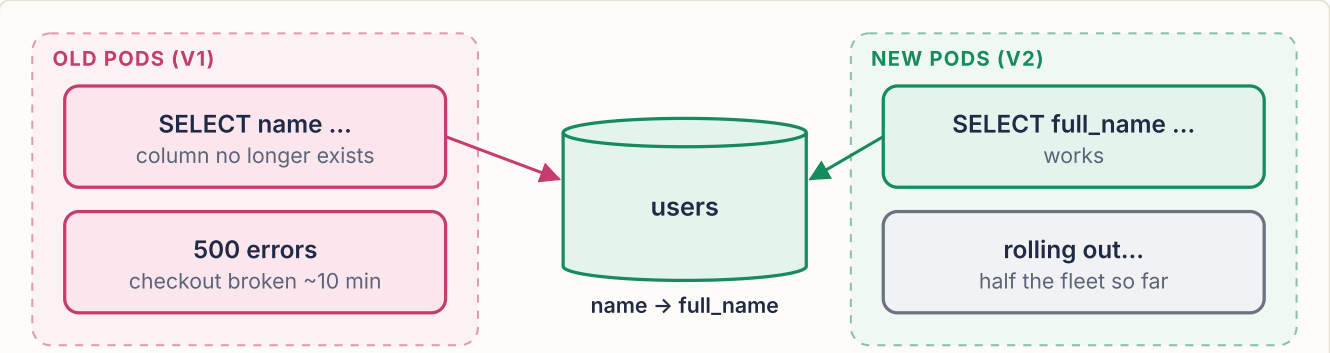


Figure 1. During a rolling deploy, old and new code run at the same time. A rename breaks every old pod at once.

2 Versioned migrations

Every schema change is a **migration file**, numbered and kept in source control next to the code. Tools: **Alembic** (Python, SQLAlchemy) and **Flyway** (Java). The tool records which migrations have run in a table inside the database (Alembic: `alembic_version`; Flyway: `flyway_schema_history`). So dev, staging and production apply the **same steps in the same order**, and no one runs ad-hoc SQL by hand.

Run migrations as a **separate deploy step**, once, before or after the app rollout. Never run them automatically at app startup: fifty pods starting together would race to run the same migration.

3 The hidden danger: locks

Many `ALTER TABLE` commands need an **exclusive lock** on the table. If a long query is already running, the ALTER waits for it. Worse, every new query on that table now queues **behind the waiting ALTER**. The whole app freezes, even though the change itself would take milliseconds.

The fix: set a short **lock timeout** (a few seconds). If the lock is not free in time, the ALTER fails fast, traffic resumes, and you retry later.

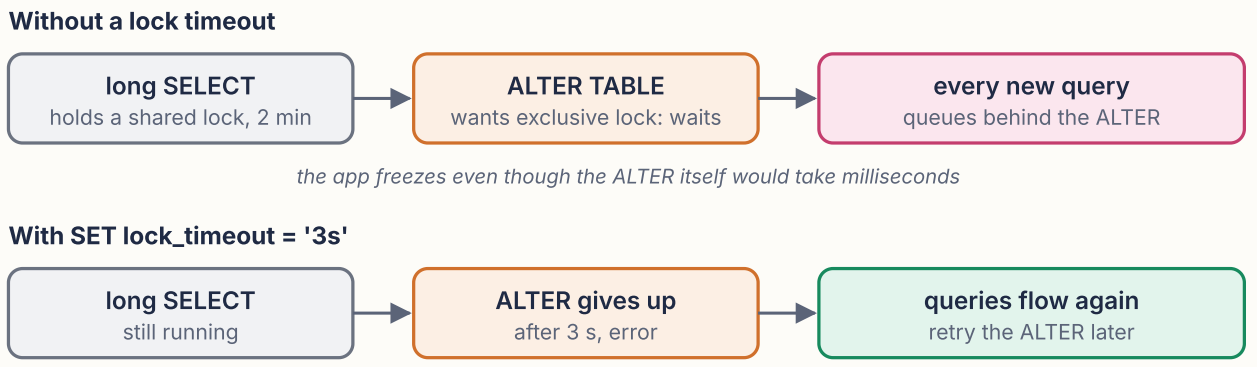


Figure 2. The ALTER is not slow; waiting for its lock is what blocks everyone. A lock timeout turns a freeze into a quick, retryable error.

Indexes are a second trap. A plain `CREATE INDEX` blocks writes to the table while it builds, which can take minutes on a big table. In Postgres, `CREATE INDEX CONCURRENTLY` builds it without blocking writes. It is slower, it **cannot run inside a transaction** (many migration tools wrap each step in one, so turn that off for this step), and if it fails it leaves an **invalid index** that you must drop before retrying.

```
SET lock_timeout = '3s';            -- fail fast instead of freezing the table
ALTER TABLE users ADD COLUMN full_name text;   -- nullable: instant

CREATE INDEX CONCURRENTLY idx_users_full_name ON users (full_name);
-- if it fails: DROP INDEX CONCURRENTLY idx_users_full_name; then retry
```

4 Expand and contract

For breaking changes such as a rename, a type change or splitting a column, use **expand and contract** (also called parallel change):

1. **Expand:** add the new column `full_name`, nullable.
2. **Dual write:** deploy code that writes to both `name` and `full_name`, but still reads `name`.
3. **Backfill:** copy existing data into `full_name`, in small batches.
4. **Switch reads:** deploy code that reads `full_name`.
5. **Stop writing** the old column.
6. **Contract:** drop `name`.

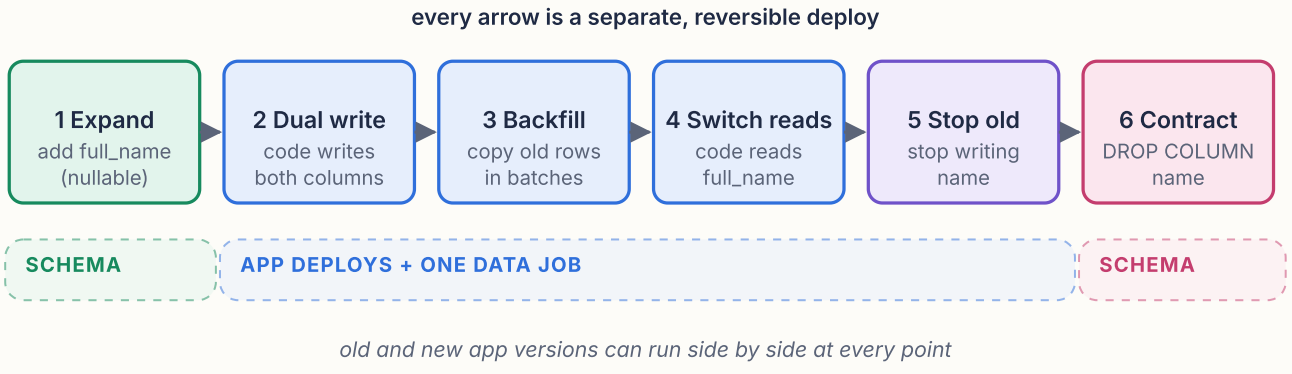


Figure 3. Six small steps instead of one big one. Schema changes bracket the code changes: expand first, contract last.

Every step is **backward compatible**, so old and new app versions can run side by side, and every step can be deployed and rolled back on its own. It takes several deploys instead of one, but users never

notice. If something goes wrong halfway, you simply stop, and the app keeps working with the old column. Each change is small, so it is easy to review, test and monitor.

5 Backfills and NOT NULL, done safely

A backfill that updates 50 million rows in one statement holds locks for a long time and floods replicas with lag. Instead: update **about 10,000 rows at a time**, commit, pause briefly, repeat. Make it **resumable** (and idempotent), so a crash restarts where it stopped instead of from the beginning.

```
-- one batch; loop over id ranges, sleep ~100 ms between batches
UPDATE users SET full_name = name
 WHERE id BETWEEN :lo AND :hi AND full_name IS NULL;

-- NOT NULL without a long blocking scan
ALTER TABLE users ADD CONSTRAINT full_name_nn
  CHECK (full_name IS NOT NULL) NOT VALID;   -- instant: only new rows checked
ALTER TABLE users VALIDATE CONSTRAINT full_name_nn; -- scans, but does not block writes
```

Adding a check constraint as **NOT VALID** is instant and applies to new writes. **VALIDATE** then checks old rows with a weaker lock that lets reads and writes continue. (In Postgres 12+, a later `SET NOT NULL` can use the validated constraint and skip its own full scan.)

WORKED EXAMPLE: BACKFILLING 50 MILLION ROWS

Batches of 10,000 rows = **5,000 batches**. If each batch takes 50 ms plus a 100 ms pause, that is 5,000 × 0.15 s = **750 s, about 12.5 minutes**. Each lock lasts only ~50 ms, replicas keep up, and if the job dies at batch 3,200 the `full_name IS NULL` filter means it simply picks up the remaining rows.

COMMON MISTAKES

- Renaming or dropping a column in the same deploy as the code change.
- Running `ALTER TABLE` with no lock timeout on a busy table.
- Plain `CREATE INDEX` on a large production table; or `CONCURRENTLY` inside a transaction.
- Running migrations in the app's startup command, so many pods race.
- One giant `UPDATE` for a backfill instead of small, resumable batches.

6 Key terms

Migration: a numbered, versioned schema change file

Alembic / Flyway: migration tools for Python / Java

Rolling deploy: replace servers a few at a time

Lock timeout: max time to wait for a lock, then fail

CONCURRENTLY: build an index without blocking writes

Expand / contract: add new, migrate, then remove old

Dual write: app writes old and new columns

Backfill: copy existing data in batches

NOT VALID: constraint checked later by VALIDATE

7 Interview questions

Q1. How do you run a schema migration with zero downtime? BASIC

Version it with Alembic or Flyway. Use expand and contract: add the new structure, dual write, backfill in batches, switch reads, stop old writes, then drop the old structure. Each step is backward compatible and deployed separately. Use lock timeouts and CREATE INDEX CONCURRENTLY to avoid long locks.

Q2. Why can't you just rename a column? BASIC

During a rolling deploy, old and new code run together for minutes. The old code still reads the old name and fails the instant the rename lands.

Q3. What does CREATE INDEX CONCURRENTLY change? BASIC

It builds the index without blocking writes. It is slower, cannot run inside a transaction, and leaves an invalid index on failure that you must drop and retry.

Q4. An ALTER TABLE froze your app for 2 minutes though it should take milliseconds. Why? SCENARIO

It waited for an exclusive lock behind a long-running query, and all new queries queued behind it. Set lock_timeout to a few seconds, retry the ALTER later, and look for long queries or idle-in-transaction sessions first.

Q5. Add a NOT NULL column to a 200M-row table safely. SCENARIO

Add it nullable (instant). Deploy code that always writes it. Backfill in batches of ~10,000 rows. Add CHECK (col IS NOT NULL) NOT VALID, then VALIDATE CONSTRAINT. Only then rely on it, optionally SET NOT NULL (Postgres 12+ skips the scan).

Q6. Halfway through expand-contract, the new read path has a bug. What now? SCENARIO

Roll back only the read-switch deploy. Both columns are still written and the old column is intact, so the old code works. Fix the bug and redeploy; nothing has been dropped.

Q7. Why not run migrations at application startup? TRICKY

With 50 pods starting together, 50 processes race to apply the same migration, and startup becomes slow and fragile. Run it once as a separate pipeline step or job.

Q8. Your backfill runs one UPDATE over all rows. What goes wrong? TRICKY

One huge transaction locks millions of rows, bloats the table, can't be resumed after a failure, and generates a burst of WAL that puts replicas minutes behind. Batch it: ~10,000 rows, commit, pause, resumable.

Next: SD-E-029 · Database Connection Pooling backendroadmap.com